

An Encrypted Network Traffic Classification Strategy: Combining Locality-Sensitive Hashing with Transformer Encoder and CNN

Junrui Wu*, Wenyong Wang*[†], Yuwei Li*, Haoran Luo*, Shuisong Hu*, Yubo Li[§]

* School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu, China

§ Product Business Group, NSFOCUS, Chengdu, China

vincentwu@std.uestc.edu.cn, wangwy@uestc.edu.cn, 202122080713@std.uestc.edu.cn,

HaoranMe@outlook.com, hi.shuisong@gmail.com, liyubo@nsfoucs.com

Abstract—Encrypted network traffic classification is fundamental for network management and security, including DDoS detection and mitigation. However, unknown category traffic can interfere with the classification of known traffic categories, leading to low recognition rates. To address this issue, we leverage the capability of Locality-Sensitive Hashing (LSH) to efficiently handle high-dimensional traffic data. By mapping similar flows into the same hash buckets with LSH, the unknown category traffic can be filtered out, thereby reducing the interference from unknown category traffic and improving the classification accuracy of known category traffic. Based on this, we propose an LSH-TECNN strategy that combines the LSH with Convolutional Neural Network (CNN) and Transformer Encoder (TE). Firstly, a combination of K-Nearest Neighbors (KNN) and LSH is used to determine whether the traffic belongs to known categories. Then, a Transformer Encoder and 2D-CNN are used to precisely classify the encrypted traffic. Experimental results demonstrate that our method improves classification accuracy by up to 12% and precision by up to 10% compared to existing encrypted traffic classification methods.

Index Terms—Encrypted Network Traffic, Traffic Classification, Locality-Sensitive Hashing, CNN, Transformer Encoder

I. INTRODUCTION

Nowadays Internet necessitates effective management to maintain optimal network performance. Consensus is that classifying diversified network traffic is a crucial step towards effective network management, which is pivotal for implementing subsequent management strategies such as quality of service (QoS) control, pricing, resource usage planning, and malware/intrusion detection [1]. Effective traffic classification not only enables proactive network adjustments and optimizations, but also serves as a cornerstone in real-world traffic engineering, ensuring that network resources are allocated efficiently and security measures are targeted precisely. Additionally, accurate traffic classification is crucial for detecting and mitigating DDoS attacks, which can disrupt network services and degrade performance [2]. However, with the increasing concern of users regarding privacy and security issues, network traffic encryption has become a crucial security measure. Encryption is vital for maintaining privacy, ensuring

that network traffic data remains secure from interception. It helps to thwart eavesdroppers from accessing passwords or monitoring application usage patterns. Today, over half of the global internet traffic is protected by encryption protocols like SSL/TLS [3]. Although encryption offers numerous benefits, it also poses significant challenges for traffic classification. Traditional traffic classification methods rely on deep inspection and analysis of packet contents, but encryption obscures these contents, rendering such methods ineffective [4]. This results in existing classification methods exhibiting lower recognition rates and accuracy when dealing with encrypted traffic. Therefore, developing new traffic classification techniques that can effectively identify and classify encrypted traffic without relying on packet contents has become a key focus of current research.

A range of technologies and solutions have been proposed to tackle the issue of classifying encrypted network traffic. These studies share a common feature: the proposed methods are designed with closed-set models, where all categories in the test data have appeared during the training process. [5]. For instance, the method proposed by Akbari et al. [6] focuses solely on classifying website protocol traffic and neglects other types of traffic, which affects the overall effectiveness of the classification. The characteristics of closed-set models make these methods perform exceptionally well on specific datasets but tend to misclassify traffic of unknown categories as known categories in real networking scenarios, leading to misclassification and security risks.

In summary, this raises a scientific question: How can we reduce the interference of unknown category traffic while ensuring the effectiveness of encrypted network traffic classification?

To address the above scientific question, we present an encrypted network traffic classification strategy called Combining Locality-Sensitive Hashing with Transformer Encoder and CNN (LSH-TECNN). Our strategy achieves encrypted network traffic classification through two stages. The first stage involves a binary classification, which aims to separate the input encrypted network traffic into two categories using LSH: unknown and known category traffic. The unknown category traffic will be set aside from further processing, while the

[†] The corresponding author.

known category traffic will serve as the input for the next stage. In the second stage, we employ a method based on TE and CNN to conduct multiclass classification on the known category traffic, achieving a more precise classification for each flow of the known category traffic.

The contributions of this paper are related to three aspects:

- 1) We propose an encrypted network traffic classification strategy that effectively classifies encrypted network traffic with unknown categories.
- 2) Our method employs KNN and LSH for binary classification to filter out unknown category traffic, followed by a TE and CNN for precise multiclass classification.
- 3) We conduct performance evaluations on two datasets, ISCX2016 and Tele2022, comparing our strategy with several existing network traffic classification methods to demonstrate its effectiveness.

II. RELATED WORK

Research on encrypted network traffic classification has recently focused on machine-learning techniques. Various approaches have been proposed to improve the accuracy and efficiency of classifying encrypted traffic, addressing challenges such as the inability to inspect packet payloads due to encryption. Below, we discuss some significant contributions in this field.

Aceto et al. [7] conducted an extensive study on deep learning methods for mobile encrypted traffic classification. They highlighted the effectiveness of CNN in automatically extracting features from traffic flows, demonstrating significant improvements over traditional methods. Xu et al. [8] introduced FastTraffic, a lightweight method for encrypted traffic classification on low-resource devices. N-gram embedding and a three-layer MLP achieved high classification accuracy and efficiency compared with state-of-the-art methods. Yan et al. [9] introduced a high-speed encrypted traffic classification (HETC) method. HETC uses payload features with aggregation entropy and chi-square tests for encryption detection, followed by a Random Forest model for fine-grained classification, achieving 94% F-measure in detecting encrypted flows. Dai et al. [10] presented CMFTC, a cross-modality fusion model for efficient multitask encrypted traffic classification in the IIoT environment. CMFTC achieves high performance with only 18% of the parameters and half the floating-point operations compared to state-of-the-art models, fusing packet, context, and flow-level modalities. Yang et al. [11] proposed DM-HNN, a hybrid neural network that uses dual-mode feature extraction for network traffic classification. The model combines flow-level and packet-level features using GRU and SAE networks, achieving excellent performance on public datasets by leveraging the joint representations of both feature types.

The discussed studies indicate that machine learning, particularly deep learning, offers promising solutions for encrypted traffic classification. Although our working also employs deep learning, it differs from the aforementioned methods in the following way: The aforementioned studies are primarily

trained and tested on closed-set datasets, failing to accurately classify unknown category traffic, which limits their real-world applicability.

III. OVERVIEW OF LSH-TECNN

To reduce the interference from unknown category traffic, and achieve encrypted network traffic classification, we have proposed an LSH-TECNN strategy. The proposed strategy can be broadly divided into two stages: KNN & LSH and TECNN. The main objective of the first stage is to quickly filter out unknown category flows using a combination of KNN and LSH. The second stage aims to achieve more precise classification of the known category flows using TECNN, ensuring that each flow to be analyzed can be matched to a known category. For clarity, in the following text, we refer to the flows of input encrypted network traffic as *flows to be detected*, the flows used in the training phase, each of which has a known category, as *closed-set flows*, and the known categories as *closed-set categories*. Additionally, a *known category flow* refers to an input flow whose category belongs to the closed-set categories, but we do not know its category until it is detected. Similarly, an *unknown category flow* refers to an input flow whose category does not belong to the closed-set categories. The overall process of LSH-TECNN is illustrated in Fig. 1.

As shown in Fig. 1, in the first stage, the flows to be detected include an unknown category flow (green block) and a known category flow (red block), while other shapes represent closed-set flows. Identical shapes in closed-set flows indicate the same category. We use KNN to find the K closest closed-set flows to the input flows. If at least one distance is within the threshold θ , the input flows are likely from closed-set categories (e.g., red block in Fig. 1). Otherwise, they are considered unknown (e.g., green block in Fig. 1). KNN reduces the number of input flows but can be error-prone due to feature value fluctuations [12]. Therefore, we employ Locality-Sensitive Hashing (LSH) to facilitate rapid feature comparison, minimizing misclassifications. Closed-set flows are assigned to hash buckets that cluster similar elements. When an input flow aligns with a bucket that clusters similar features, it is likely associated with a known category. This approach is particularly effective for large datasets, where the efficiency of LSH in approximating nearest neighbors without a significant loss in accuracy is notably enhanced. After stage one, the input flows are considered to have known categories. Although LSH provides approximate flow matching, due to the complexity of the network environment and the variability of encrypted traffic, we need to use TECNN for more precise flow classification. In stage 2, TECNN is utilized for precise classification. The Encoder module can extract global context information from the input flows and capture dependencies between the flows. The CNN module extracts local features from the flow representation, including global context information.

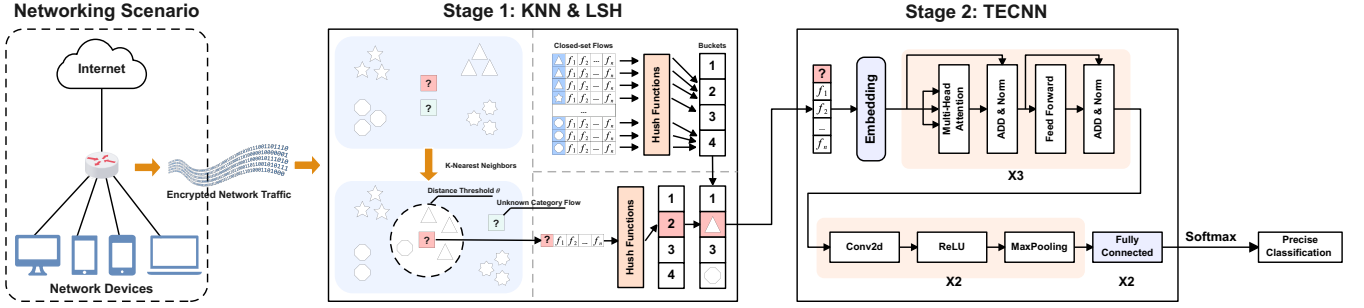


Fig. 1. The Overall Process of LSH-TECNN.

IV. METHODS

A. Stage 1: Unknown Category Traffic Filtering

The input traffic, composed of known and unknown category traffic, consists of multiple flows. Each flow comprises packets that share the same five-tuple information and maintains a uniform duration. Through feature extraction software, we can extract consistent statistical features from each flow, enabling thorough data analysis and processing. In Stage 1, the unknown category traffic filtering, which comprises KNN and LSH, evaluates each flow to determine whether it belongs to a known or unknown category. If it is a known category, the pre-trained model will precisely classify it in stage 2. If not, the flow is saved to a file database for further analysis.

Our method's unknown category traffic filter first employs KNN to quickly identify the closed-set flows similar to each input flow by finding the closest K closed-set flows using Euclidean distance. The flow is classified as likely known if these distances are within a specified threshold θ .

Assuming that the closed-set flows $T_{closed} = \{f_1 : (x_1, l_1), f_2 : (x_2, l_2), \dots, f_j : (x_j, l_j), \dots, f_M : (x_M, l_M)\}$, where $x_j \in X \subseteq \mathbb{R}^n$ is the feature vector of each flow, $l_j \in C = \{c_1, c_2, \dots, c_N\}$ is the category of each flow, and M is the number of flows contained in the T_{closed} . For each input flow I_i to be detected, we first calculate the Euclidean distance between I_i and each flow in T_{closed} , which can be shown as:

$$d(I_i, f_j) = \sqrt{\sum_{k=1}^n (I_{ik} - f_{jk})^2}, f_j \in T_{closed}, \quad (1)$$

where I_{ik} and f_{jk} represent the k -th feature of the input flow I_i and the j -th flow in T_{closed} , respectively, and n is the number of features. Let $d(I_i, T_{closed})$ denote the set of Euclidean distances between I_i and each flow in T_{closed} . We first select the points with distances less than the threshold θ , and then choose the K nearest points from these, as illustrated by the following equation:

$$d_\theta = \{d_i \mid d_i \leq \theta, d_i \in d(I_i, T_{closed})\}, \quad (2)$$

$$N_K(I_i) = \{f_j \mid d(I_i, f_j) \in d_\theta(K)\}, \quad (3)$$

where $N_K(I_i)$ represents the closed-set flows that are nearest to the input flow I_i in terms of Euclidean distance. $d_\theta(K)$

denotes the set of the nearest K elements $d(I_i, f_j)$ that are less than the threshold θ . If for any input flow I_i , $N_K(I_i) = \emptyset$, it is considered as unknown traffic; otherwise, it proceeds to the next step of processing.

Due to the fluctuation in the feature values of flows, errors may occur. Specifically, a flow may have a Euclidean distance less than θ but still be an unknown category, or it may have a Euclidean distance greater than θ with all closed-set flows but still be a known category. Therefore, we utilize LSH for rapid feature comparison to reduce false positives.

LSH is a fast nearest neighbor search algorithm for massive high-dimensional data. The fundamental role of LSH is to efficiently handle the nearest neighbor problem in massive high-dimensional data. When $N_K(I_i) \neq \emptyset$, we use LSH to determine if I_i has similar flows. First, we define a hash function family $H = \{h_1, h_2, \dots, h_H\}$, and each hash function in H can be defined as follow:

$$h(u) = \left\lfloor \frac{\langle u, \vec{v}_h \rangle + t_h}{T} \right\rfloor, \quad (4)$$

where T is a fixed value used for all h in H , $\vec{v}_h \sim N^d(0, 1)$ is a vector randomly chosen from the normal distribution, and $t_h \in [0, T]$ is a value randomly selected from the interval $[0, T]$.

After constructing the hash function, we hash the closed-set flows. For each closed-set flow f_j , we apply the hash function h to map it to a bucket in each of the hash tables. In our method, we determine the number of hash tables L , the number of hash functions τ within each table.

For a given input flow I_i , where $N_K(I_i) \neq \emptyset$, we hash it using the same set of hash functions, as shown as:

$$B_{I_i} = [H_\tau^1(I_i), H_\tau^2(I_i), \dots, H_\tau^L(I_i)], \quad (5)$$

where B_{I_i} represents the bucket to which flow I_i is mapped in each hash table, and $H_\tau^L(I_i)$ represents the processing of flow I_i using τ hash functions in hash table L .

To determine the similarity between the input flow I_i and the closed-set flows, we check the buckets in which the input flow I_i is mapped. If there are closed-set flows in the buckets where I_i is mapped, and the closed-set flows retrieved from these buckets are the same across multiple tables, then I_i is

considered similar to the closed-set flows. Otherwise, I_i is considered to be unknown traffic.

B. Stage 2: Precise Classification

After the unknown category traffic filtering, we obtain the known category traffic. However, we only know that those flows belong to the closed-set categories, without knowing their more precise categories. Therefore, in the second stage, we proposed an encrypted traffic classification model named TECNN to classify the known category traffic more precisely.

To optimize the efficiency of data processing and analysis in our approach, we will extract the packet length and window value from each packet within each filtered flow, constructing sequences of packet lengths $ps = \{p_1, p_2, \dots, p_d\}$ and window values $ws = \{w_1, w_2, \dots, w_d\}$. These sequences will subsequently be combined into a data sequence $X = \{x_1, x_2, \dots, x_d\}$, which will serve as input for stage 2.

The structure of the TECNN is shown in the right part of Fig.1. The model consists of a Transformer Encoder module and a 2D-CNN module, combining the ability of the Transformer to capture global context information with the capability of the CNN to extract local spatial features.

The Transformer Encoder module consists of Multi-Head Self-Attention, Residual Connection, Layer Normalization, and Feed Forward Neural Network.

First, each element in the input data sequence is transformed into a continuous vector form after passing through the embedding layer. Suppose the input data is $X = \{x_1, x_2, \dots, x_d\}$, where each element x_i in X is mapped into an e -dimensional vector through the embedding module, then the output after embedding $E \in \mathbb{R}^{d \times e}$ is as follows:

$$E = \begin{bmatrix} E_{1,1} & E_{1,2} & \cdots & E_{1,e} \\ E_{2,1} & E_{2,2} & \cdots & E_{2,e} \\ \vdots & \vdots & \ddots & \vdots \\ E_{d,1} & E_{d,2} & \cdots & E_{d,e} \end{bmatrix}. \quad (6)$$

In the Multi-Head Self-Attention mechanism, an important process is the calculation of Scaled Dot-Product Attention, which is expressed as follows:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V, \quad (7)$$

When the input data E is processed by the Multi-Head Self-Attention mechanism, it first needs to undergo linear transformations using three learnable weight matrices W^Q , W^K , and W^V , generating Q (Query), K (Key), and V (Value) matrices, respectively. The weight matrices W^Q , W^K , and W^V are defined as $W^Q, W^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $W^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$, where $d_k = d_v = d_{\text{model}}/h$, h represents the number of heads, and d_{model} is equivalent to the embedding dimension e mentioned above.

Next, by performing matrix operations on Q and K , we can obtain the attention scores for each item of data with other items of data. The attention scores are used to represent the relevance between different elements in the sequence. This relevance allows each element to incorporate information from

other elements, thereby achieving the goal of extracting global context information.

To reduce the issues of network degradation and vanishing gradients that occur during the training of deep networks and to improve model training stability, the Transformer introduces residual connection and layer normalization.

After the input data X is processed by the Encoder module, a data representation with rich global contextual dependency is obtained.

Although the Transformer Encoder can capture the global dependencies of the input data, it lacks the ability to extract local spatial features. Therefore, we incorporate the 2D-CNN module as the second submodule of TECNN to obtain key local features from the global context information. The 2D-CNN module consists of two 2D convolutional layers, two max-pooling layers, and two fully connected neural networks. The input and output data dimensions of each layer are shown in Table I, where N denotes the batch size of the input data, C represents the number of classification categories.

TABLE I
HYPERPARAMETERS OF 2D-CNN

Network Layer	Input Dimensions	Output Dimensions	Kernel Size	Stride	Padding
Conv2d+Relu	(N, 1, 64, 128)	(N, 64, 32, 64)	(2, 2)	2	0
MaxPool2d	(N, 64, 32, 64)	(N, 64, 16, 32)	(2, 2)	2	0
Conv2d+Relu	(N, 64, 16, 32)	(N, 128, 8, 16)	(2, 2)	2	0
MaxPool2d	(N, 128, 8, 16)	(N, 128, 4, 8)	(2, 2)	2	0
Linear	(N, 4096)	(N, 1024)	-	-	-
Linear	(N, 1024)	(N, C)	-	-	-
Softmax	(N, C)	(N, C)	-	-	-

First, 2D convolution is applied to the input data. Multiple filters slide over the input data to perform local convolutions, with each filter focusing on small regions to extract local features. The ReLU activation function is then used to map negative values to zero, reducing the risk of model overfitting and addressing the vanishing gradient problem. After the convolution layers, max-pooling layers are used to reduce the dimensionality of the feature maps while retaining the most significant features. This process helps to capture invariant features. The convolved data is then passed through two fully connected layers to convert the data dimensions to match the number of classification categories. Finally, the Softmax function is applied to compute the predicted probabilities for each category.

$$p_c = \text{Softmax}(z_c) = \frac{e^{z_c}}{\sum_{c=1}^N e^{z_c}}, \quad (8)$$

where p_c represents the probability of the model predicting the flow to category c , where z_c is the c^{th} component of the output vector of output layer, and N is the number of categories. The index of the maximum probability is then used as the predicted category output.

C. The Computation Complexity of LSH-TECNN

Assuming the number of input data is n , with each data entry having d input features. In the first stage, the computational complexity of KNN is $O(ndk)$, where k is a hyper-parameter

of KNN. The computational complexity of LSH can be considered as $O(nL\tau) \times O(\text{hash_function})$, where L represents the number of hash tables and τ is the number of hash functions in each hash table. In our strategy, the hash function is defined as shown in formula 4, with a computational complexity of $O(d)$. The computational complexities of TE and CNN are considered as $O((1+h^2) \cdot n^2 \cdot d)$ and $O(n \cdot c)$ respectively, where c represents the number of known categories, and h is the number of attention heads. Therefore, the overall computational complexity is $O(nL\tau d + nc + (1+h^2) \cdot n^2 \cdot d)$. Since L , τ , and c are much smaller than n , the computational complexity can be approximated as $O((1+h^2) \cdot n^2 \cdot d)$.

V. EXPERIMENTS

A. Experimental Environment

We conducted experiments on a computer with an Intel Core i7-11700 CPU, Nvidia RTX 3060 GPU and 32GB of RAM. The datasets we used are ISCX2016 [13] and Tele2022, from which 6,500 and 8,600 flows were selected, respectively. All selected flows have been stripped of features such as IP addresses, ports, and protocols that could lead to insufficient generalization of the algorithm. We selected some categories of data as unknown category traffic, and the remaining data as closed-set flows. The known category traffic was divided into training, validation, and test sets in a ratio of 6:2:2. The parameters are set as follows: $K = 10$ and $\theta = 0.5$ for the distance metrics, with $L = 5$ hash tables utilized, each containing $\tau = 20$ hash functions.

B. Experimental Results

Firstly, we evaluate the unknown category traffic filtering capability of the LSH-TECNN strategy, as shown in Fig. 2. Whether ISCX2016 or Tele2022, KNN&LSH outperforms KNN in all performance metrics. This result indicates that using only KNN to filter unknown category traffic cannot eliminate the fluctuation in the feature values of flows. Furthermore, KNN&LSH performs better on Tele2022 than on ISCX2016 because the data in Tele2022 mainly consists of network videos, which are similar to each other, whereas ISCX2016 contains a variety of data categories with significant differences.

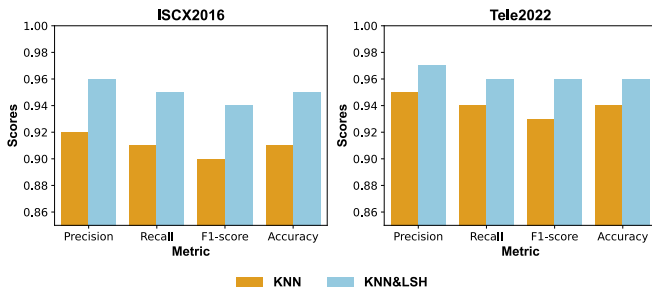


Fig. 2. Metrics of KNN and KNN&LSH of Unknown Category Traffic Filtering.

From the confusion matrix in Fig. 3, it can be observed that when the model has the capability to identify unknown

categories, it can recognize and categorize them as unknown categories. Additionally, it can be seen that some flows of known categories are misclassified as unknown categories. The reason these flows are identified as unknown categories is that their feature value fluctuations exceed the set threshold.

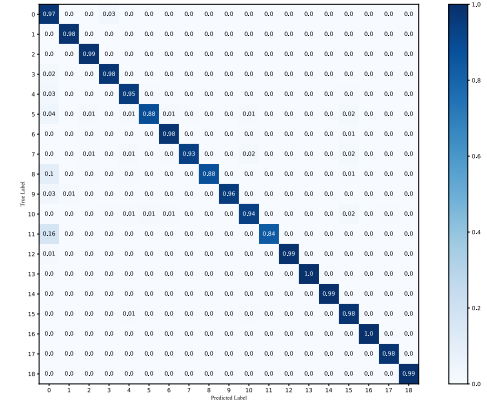


Fig. 3. Confusion Matrix for Unknown Category Traffic Filtering on Tele2022.

After unknown category traffic filtering, we test the performance metrics of TECNN on two datasets with known category traffic. Due to space limitations, we only present the multi-class experimental results on ISCX2016, as shown in Table II. It can be seen that the performance metrics for most categories exceed 90%. However, the accuracy for a few categories, such as 1 (AIM) and 5 (ICQ), is relatively low. This is because the traffic for these categories is inherently limited, and both are chat applications with similar feature value ranges, such as packet length, leading to a higher probability of misclassification.

TABLE II
DETAILED PERFORMANCE METRICS OF LSH-TECNN ON THE ISCX2016 DATASET

	Precision	Recall	F1-score	Percentage (%)
1	0.767	0.971	0.857	0.0003
2	0.971	0.954	0.963	0.0023
3	0.990	0.934	0.961	0.0834
4	0.999	0.999	0.999	0.3450
5	0.759	0.953	0.845	0.0006
6	0.991	0.993	0.992	0.0935
7	0.933	0.966	0.949	0.0005
8	0.997	0.998	0.998	0.0646
9	0.908	1.000	0.952	0.0272
10	0.997	0.999	0.998	0.0534
11	0.978	0.993	0.985	0.1538
12	0.974	0.977	0.975	0.0087
13	0.998	0.994	0.996	0.0296
14	0.995	0.987	0.991	0.0259
15	0.998	1.000	0.999	0.0857
16	0.988	0.990	0.989	0.0255

As shown in Fig. 4, we compared LSH-TECNN with other encrypted network traffic classification algorithms. It can be seen that our method achieves superior performance on both datasets, primarily because it filters out unknown category traffic. In contrast, other methods tend to misclassify

unknown category traffic as known categories. Additionally, Deep Packet also shows promising results; however, its longer runtime makes it less efficient compared to our method.

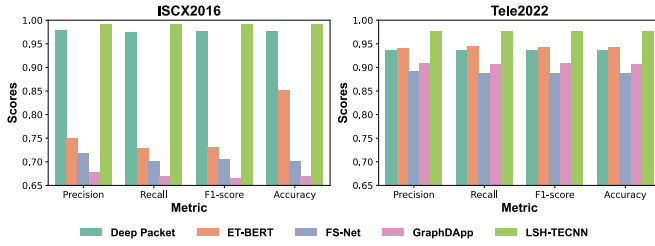


Fig. 4. Performance Comparison With Other Methods.

To validate the effectiveness of the LSH-TECNN strategy, ablation experiments were conducted on the two submodules used: Encoder and CNN. As shown in Fig. 5, on the ISC2016 dataset, the combination of Encoder and CNN achieved the best performance metrics. Our method improved accuracy by 0.64%, precision by 0.6%, recall by 0.64%, and F1-score by 0.63% compared to CNN. Additionally, the performance metrics of accuracy, recall, precision, and F1-score were all higher for the Encoder alone than for CNN, indicating that the Encoder is more suitable for handling time-series data. On the Tele2022 dataset, the performance of the LSH-TECNN strategy remains optimal and is even more pronounced than on the ISC2016 dataset. Compared to the CNN model, TECNN improved accuracy by 2.89%, recall by 2.82%, precision by 3.1%, and F1-score by 2.89%. Notably, the performance of the Encoder alone also surpassed that of the CNN, with accuracy higher by 1.8%, recall by 1.72%, precision by 1.8%, and F1-score by 1.78%. The efficiency improvement is mainly due to Stage 1 filtering out the unknown category traffic, reducing the number of unknown traffic assigned to known categories in Stage 2 by TECNN. In contrast, other methods lack a mechanism to filter unknown category traffic.

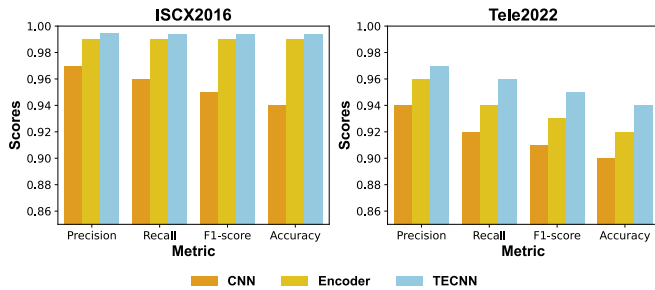


Fig. 5. Metrics of KNN and KNN&LSH of Unknown Category Traffic Filtering.

VI. CONCLUSION

In this paper, we proposed LSH-TECNN, a novel strategy for encrypted network traffic classification. Our two-stage method addresses the challenge of unknown category traffic interference. The first stage uses K-Nearest Neighbors

(KNN) and locality-sensitive hashing (LSH) to filter unknown traffic, ensuring that only known categories proceed to the next stage. The second stage employs TECNN to classify known traffic with high precision. Experimental results on ISC2016 and Tele2022 datasets show that LSH-TECNN outperforms other classification methods in various performance metrics, including accuracy, precision, recall, and F1-score. Our strategy's ability to effectively filter unknown traffic categories makes it particularly suitable for enhancing network security against DDoS attacks. Moreover, the combination of KNN and LSH provides a robust mechanism for preliminary filtering, significantly reducing the complexity of subsequent classification tasks. Future work will focus on developing and integrating advanced feature extraction techniques specifically tailored for DDoS attack mitigation, aiming to enhance the system's performance and resilience against these evolving cyber threats.

REFERENCES

- [1] S. Rezaei and X. Liu, "Deep learning for encrypted traffic classification: An overview," *IEEE Communications Magazine*, vol. 57, no. 5, pp. 76–81, 2019.
- [2] W. Jiang, H. Li, G. Xu, T. Zhang, and R. Lu, "A comprehensive defense framework against model extraction attacks," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 2, pp. 685–700, 2023.
- [3] M. Shen, Y. Liu, L. Zhu, K. Xu, X. Du, and N. Guizani, "Optimizing feature selection for efficient encrypted traffic classification: A systematic approach," *IEEE Network*, vol. 34, no. 4, pp. 20–27, 2020.
- [4] H. Tahaei, F. Afifi, A. Asemi, F. Zaki, and N. B. Anuar, "The rise of traffic classification in iot networks: A survey," *JOURNAL OF NETWORK AND COMPUTER APPLICATIONS*, vol. 154, MAR 15 2020.
- [5] M. Shen, K. Ye, X. Liu, L. Zhu, J. Kang, S. Yu, Q. Li, and K. Xu, "Machine learning-powered encrypted network traffic analysis: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 1, pp. 791–824, 2023.
- [6] I. Akbari, M. A. Salahuddin, L. Ven, N. Limam, R. Boutaba, B. Mathieu, S. Moteau, and S. Tuffin, "Traffic classification in an increasingly encrypted web," *Commun. ACM*, vol. 65, no. 10, p. 7583, sep 2022. [Online]. Available: <https://doi.org/10.1145/3559439>
- [7] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescap, "Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges," *IEEE Transactions on Network and Service Management*, vol. 16, no. 2, pp. 445–458, 2019.
- [8] Y. Xu, J. Cao, K. Song, Q. Xiang, and G. Cheng, "Fasttraffic: A lightweight method for encrypted traffic fast classification," *Computer Networks*, vol. 235, p. 109965, 2023.
- [9] X. Yan, L. He, Y. Xu, J. Cao, L. Wang, and G. Xie, "High-speed encrypted traffic classification by using payload features," *Digital Communications and Networks*, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2352864824000208>
- [10] J. Dai, X. Xu, H. Gao, and F. Xiao, "Cmftc: Cross modality fusion efficient multitask encrypt traffic classification in iiot environment," *IEEE Transactions on Network Science and Engineering*, vol. 10, no. 6, pp. 3989–4009, 2023.
- [11] Y. Yang, Y. Yan, Z. Gao, L. Rui, R. Lyu, B. Gao, and P. Yu, "A network traffic classification method based on dual-mode feature extraction and hybrid neural networks," *IEEE Transactions on Network and Service Management*, vol. 20, no. 4, pp. 4073–4084, 2023.
- [12] J. Huang, Y. Wei, J. Yi, and M. Liu, "An improved knn based on class contribution and feature weighting," in *2018 10th International Conference on Measuring Technology and Mechatronics Automation (ICMTMA)*, 2018, pp. 313–316.
- [13] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and vpn traffic using time-related," in *Proceedings of the 2nd international conference on information systems security and privacy (ICISSP)*, 2016, pp. 407–414.